JAVA INTERVIEW HANDBOOK

Modules 1–3 • Beginner → Interview Ready

Day 1 — Java Core I: OOP & Strings
Day 2 — Java Core II: Collections & Exceptions
Day 3 — Java Core III: Multithreading & Java 8

Purpose

Build the concepts from first principles, understand what the JVM is doing, write small practical programs, and develop short interview-ready explanations.

How to Use This Handbook

Do not memorize the interview answers before understanding the mechanism. For each topic, first read the beginner explanation, then type the code yourself, then close the notes and explain the concept aloud. The final checkpoint for each day is designed to test whether you can reason without depending on memorized sentences.

Foundation	Java, JDK/JRE/JVM, compilation, bytecode, JIT, memory basics
Module 1 / Day 1	OOP, constructors, this/super, static/final, strings
Module 2 / Day 2	Collections, HashMap internals, iterators, exceptions
Module 3 / Day 3	Threads, synchronization, concurrency, Java 8 features
Final drills	Interview definitions, traps, coding checkpoints

The 90-second rule

For a basic interview question, your answer should follow: **definition** → **why** → **simple example** → **one important distinction**. Do not try to fill 90 seconds. A clear 30–60 second answer is usually stronger than a long, unfocused one.

FOUNDATION — JAVA FROM THE GROUND UP

1. What is Java?

Definition: Java is a high-level, class-based, object-oriented programming language whose compiled bytecode runs on a JVM.

Why it matters: It explains why the same Java program can run on different operating systems when a suitable JVM is available.

Beginner explanation: You write source code in a .java file. The Java compiler converts it into bytecode in a .class file. The JVM loads and executes that bytecode and may use JIT compilation to turn frequently executed code into native machine instructions.

Practical example:

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello Java");
    }
}
```

> **Interview answer:** Java is a high-level, class-based, object-oriented language. Its source code is compiled into platform-independent bytecode, and the JVM executes that bytecode on the target platform.

2. JDK, JRE and JVM

Definition: JVM executes bytecode; JRE provides the JVM plus runtime libraries; JDK provides the development tools plus the runtime needed to build and run Java applications.

Why it matters: Interviewers use this to check whether you understand the Java execution environment rather than treating Java as simply a compiler.

Beginner explanation: Think of the JDK as the developer's complete toolkit. The JVM is the engine that executes bytecode. The JRE is the runtime environment around the JVM and Java libraries.

> **Interview answer:** JDK is for developing Java applications, JRE is the runtime environment, and JVM is the virtual machine that executes Java bytecode.

3. Compilation and runtime

Definition: Compile time is when source code is checked and compiled; runtime is when the compiled program is actually executed.

Why it matters: It helps you distinguish errors such as type errors from exceptions that occur while the program is running.

Beginner explanation: For example, assigning a String to an int is rejected by the compiler. Dividing an integer by zero can compile successfully but fail during execution.

Practical example:

```
int x = "hello";       // compile-time error

int a = 10;
int b = 0;
System.out.println(a / b); // runtime ArithmeticException
```

> **Interview answer:** At compile time, javac converts valid source code to bytecode and catches compile-time errors. At runtime, the JVM executes that bytecode, where runtime exceptions can occur.

4. JIT compiler

Definition: JIT, or Just-In-Time compilation, is a JVM optimization that compiles frequently executed bytecode into native machine code during program execution.

Why it matters: It explains how Java can combine portability with strong runtime performance.

Beginner explanation: The JVM does not necessarily translate every instruction from bytecode to machine code in exactly the same way every time. Hot code can be optimized and compiled into machine code so repeated execution becomes faster.

> **Interview answer:** JIT is a JVM component that identifies frequently executed code and compiles it to native machine code at runtime for better performance.

5. Stack and Heap — basic model

Definition: The stack stores method-call frames and local variables/references, while the heap stores objects and arrays.

Why it matters: You need this model for objects, strings, collections and multithreading.

Beginner explanation: If you create an object, the object itself is generally on the heap, while a local reference variable can live in the current stack frame. This is a simplified conceptual model; JVM implementations have optimization details that can change physical placement.

Practical example:

```
class Person {
    String name;
}

Person p = new Person();
p.name = "Mekha";
```

> **Interview answer:** In a simplified JVM memory model, the local reference p is associated with the current stack frame and the Person object is on the heap. The important distinction is that a reference points to an object; the variable is not the object itself.

MODULE 1 — DAY 1: JAVA CORE I

Order: Classes/objects → four pillars → abstract class/interface → overloading/overriding → constructors → this/super → static/final → strings.

1. Classes and Objects

Definition: A class is a blueprint that defines state and behavior; an object is a runtime instance created from that class.

Why it matters: OOP concepts are built around grouping data and behavior into objects.

Beginner explanation: A class defines fields and methods. When you use new, you create an object. Different objects created from the same class can hold different state.

Practical example:

```
class Car {
    String color;

    void drive() {
        System.out.println(color + " car is driving");
    }
}

Car c1 = new Car();
c1.color = "Red";
c1.drive();
```

> **Interview answer:** A class is a blueprint containing fields and methods. An object is an instance of that class with its own state.

2. Encapsulation

Definition: Encapsulation means bundling data and the methods that operate on it, while controlling direct access to internal state.

Why it matters: It prevents outside code from freely putting an object into an invalid state.

Beginner explanation: A common Java pattern is private fields plus public methods such as getters, setters, or domain-specific operations. The class can validate values before changing its state.

Practical example:

```
class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }

    public double getBalance() {
        return balance;
    }
}
```

> **Interview answer:** Encapsulation combines state and behavior in a class and restricts direct access to internal data, commonly using private fields and controlled public methods.

3. Inheritance

Definition: Inheritance allows a class to derive fields and behavior from another class using extends.

Why it matters: It represents an is-a relationship and allows common behavior to be reused or specialized.

Beginner explanation: A child class inherits accessible members from its parent and can add new behavior or override inherited methods.

Practical example:

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

Dog d = new Dog();
d.eat();
d.bark();
```

> **Interview answer:** Inheritance lets a subclass acquire accessible state and behavior from a superclass. In Java, a class can directly extend one class.

4. Polymorphism

Definition: Polymorphism means one interface/reference can represent different concrete object behaviors.

Why it matters: It lets code depend on a general type while the actual object's overridden method determines behavior at runtime.

Beginner explanation: The most important interview example is runtime polymorphism through method overriding.

Practical example:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

Animal a = new Dog();
a.sound();   // Bark
```

> **Interview answer:** Polymorphism allows a superclass reference or interface reference to refer to different implementations. With overriding, the actual object's method is selected at runtime.

5. Abstraction

Definition: Abstraction means exposing essential behavior while hiding implementation details.

Why it matters: It lets users of a component focus on what it does instead of how it does it.

Beginner explanation: Java supports abstraction mainly through abstract classes and interfaces. A caller can use the abstraction without knowing the implementation.

Practical example:

```
interface Payment {
    void pay(double amount);
}
```

```
class UPI implements Payment {
    public void pay(double amount) {
        System.out.println("Paid " + amount);
    }
}

Payment p = new UPI();
p.pay(500);
```

> **Interview answer:** Abstraction exposes required behavior through an abstract class or interface while hiding implementation details.

6. Abstract Class vs Interface

Aspect	Abstract class	Interface
Purpose	Shared base with common state/behavior	Contract/capability
Inheritance	A class extends one class	A class can implement multiple interfaces
Fields	Can have instance fields	Fields are implicitly public static final
Constructors	Can have constructors	No constructors
Methods	Can have abstract and concrete methods	Can have abstract, default, static and private methods

> **Interview answer:** Use an abstract class when closely related classes need shared state or implementation. Use an interface when you want to define a contract or capability that multiple unrelated classes can implement.

Example: An abstract Vehicle could contain shared fields such as speed; an interface such as Flyable can be implemented by a Bird, Drone, or Aircraft.

7. Method Overloading

Definition: Overloading means having multiple methods with the same name but different parameter lists in the same class.

Why it matters: It provides compile-time polymorphism and makes related operations convenient to call.

Beginner explanation: The return type alone cannot distinguish overloaded methods. The parameter types, count, or order must differ.

Practical example:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}
```

> **Interview answer:** Overloading is compile-time polymorphism where methods share a name but have different parameter lists.

8. Method Overriding

Definition: Overriding occurs when a subclass provides its own implementation of an inherited instance method with the same signature.

Why it matters: It enables runtime polymorphism.

Beginner explanation: The key idea is to understand the rule, then observe it in the code example.

Practical example:

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Meow");
    }
}
```

Interview answer: Overriding is runtime polymorphism in which a subclass supplies a specialized implementation of an inherited method.

Overloading vs overriding — memory rule: overloading = same class/name, different parameters, compile time; **overriding = child changes inherited method, runtime dispatch**.

9. Constructors

Definition: A constructor is a special member used when an object is created; it has the same name as the class and no return type.

Why it matters: Constructors establish a valid initial state.

Beginner explanation: If you do not declare any constructor, Java provides a default no-argument constructor. Once you declare a constructor, Java does not automatically add that default constructor.

Practical example:

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }
}

Student s = new Student("Mekha");
```

Interview answer: A constructor initializes a newly created object. It has the class name and no return type. Constructors can be overloaded.

10. this

Definition: this is a reference to the current object.

Why it matters: It resolves name conflicts and lets an instance method or constructor refer to the current object.

Beginner explanation: When a constructor parameter has the same name as an instance field, this makes it explicit that the field belongs to the current object.

Practical example:

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    void show() {
        System.out.println(this.name);
    }
}
```

Interview answer: this refers to the current object. A common use is distinguishing an instance field from a constructor parameter with the same name.

11. super

Definition: super refers to the immediate superclass portion of the current object.

Why it matters: It is used to access superclass members and invoke a superclass constructor.

Beginner explanation: A child can use super to refer to a parent field or method when a member is hidden or overridden.

Practical example:

```
class Animal {
    String name = "Animal";
}

class Dog extends Animal {
    String name = "Dog";

    void show() {
        System.out.println(this.name);
        System.out.println(super.name);
    }
}
```

Interview answer: super refers to the immediate parent class and can access parent members or call a parent constructor.

12. static

Definition: static makes a member belong to the class rather than to each individual object.

Why it matters: Use it for data or behavior that should be shared at class level rather than duplicated per instance.

Beginner explanation: A static field has one class-level copy shared by instances. A static method has no current-object reference (this).

Practical example:

```
class Counter {
    static int count = 0;

    Counter() {
        count++;
    }
}

new Counter();
new Counter();

System.out.println(Counter.count); // 2
```

Interview answer: A static member belongs to the class rather than a particular object. A static field is shared among instances of that class.

Important: A static method cannot directly access instance fields because it does not have a particular object represented by this.

13. final

Definition: final prevents further modification in the context where it is applied.

Why it matters: It is used to enforce constants, prevent method overriding, or prevent class inheritance.

Beginner explanation: The meaning depends on what is final: a variable cannot be reassigned, a method cannot be overridden, and a class cannot be extended.

Practical example:

```
final int MAX = 100;
```

```
final class Utility {
}

class Parent {
    final void show() {
        System.out.println("Fixed");
    }
}
```

Interview answer: A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.

MODULE 1 — STRINGS

14. String Pool

Definition: The String Pool is the JVM-managed pool of canonical String objects for string literals and interned strings.

Why it matters: Strings are immutable, so the JVM can safely reuse identical pooled strings and reduce duplicate objects.

Beginner explanation: With a literal, Java checks the pool and reuses an existing object if the same value is already present. With new String(...), a new String object is explicitly created.

Practical example:

```
String a = "Java";
String b = "Java";

System.out.println(a == b);       // true

String c = new String("Java");
System.out.println(a == c);       // false
System.out.println(a.equals(c));  // true
```

Interview answer: The String Pool stores canonical String values so identical literals can be shared. String literals use the pool, while new String creates a distinct object.

Key mental model: a variable such as a holds a reference; it does not contain the String object itself. With String b = a;, b receives the same reference.

15. String Immutability

Definition: String is immutable: once a String object is created, its character sequence cannot be changed.

Why it matters: Immutability makes Strings safe to share, helps String Pool reuse, and makes String behavior predictable.

Beginner explanation: An operation that appears to modify a String actually produces another String value and may reassign the reference variable.

Practical example:

```
String a = "Java";
String b = a;

a = "Python";

System.out.println(a); // Python
System.out.println(b); // Java
```

Interview answer: String is immutable, so existing String objects cannot be changed. Reassignment changes which object a reference points to; it does not mutate the original String.

16. == vs equals()

Definition: For object references, == checks whether two references identify the same object; equals() checks logical equality according to the class implementation.

Why it matters: This distinction prevents subtle bugs, especially with Strings and collections.

Beginner explanation: For String, equals is overridden to compare character content.

Practical example:

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a == b);       // false
```

```
System.out.println(a.equals(b));  // true
```

Interview answer: For String objects, == compares references while equals() compares character content.

Interview trap: Never say == always compares values. For primitives it compares values; for object references it compares references.

17. String vs StringBuilder vs StringBuffer

Definition: String is immutable; StringBuilder and StringBuffer are mutable character sequences designed for repeated modification.

Why it matters: Repeated String concatenation can create many temporary String objects. Mutable builders are better for constructing text through many changes.

Beginner explanation: StringBuilder is normally preferred for single-threaded construction. StringBuffer synchronizes its methods.

Practical example:

```
String s = "A";
s = s + "B";
s = s + "C";

StringBuilder sb = new StringBuilder("A");
sb.append("B");
sb.append("C");

System.out.println(sb); // ABC
```

Interview answer: String is immutable. StringBuilder is mutable and generally preferred for repeated string construction in a single-threaded context. StringBuffer is mutable and synchronizes its methods.

Type	Mutable?	Synchronization	Typical use
String	No	Not applicable	Fixed text/value
StringBuilder	Yes	Not synchronized	Fast single-threaded building
StringBuffer	Yes	Synchronized methods	Use when synchronized String-buffer semantics are specifically requ

Module 1 — Quick Interview Drill

1. Explain the four OOP pillars using one practical example each.
2. When would you choose an abstract class instead of an interface?
3. Explain overloading vs overriding in one minute.
4. Why can't a static method directly use an instance field?
5. What does final mean for a variable, method and class?
6. Explain this and super with a constructor example.
7. Why does String need to be immutable?
8. Predict: String a="Java"; String b=new String("Java"); compare == and equals().
9. Why is StringBuilder usually preferred for repeated concatenation?

MODULE 2 — DAY 2: COLLECTIONS & EXCEPTIONS

A collection is a data structure provided by the Java Collections Framework. Choosing one depends on ordering, uniqueness, lookup speed, memory, and concurrency requirements.

1. ArrayList

Definition: ArrayList is a resizable-array implementation of List.

Why it matters: It provides fast random access by index and is a common default List choice.

Beginner explanation: It uses an array-like backing structure. When capacity is insufficient, a larger array is allocated and elements are copied.

Practical example:

```
List<String> names = new ArrayList<>();

names.add("A");
names.add("B");

System.out.println(names.get(0)); // A
names.remove(1);
```

Interview answer: ArrayList is backed by a dynamically resized array. get by index is O(1) on average, while insertion/removal in the middle is O(n).

2. LinkedList

Definition: LinkedList is a doubly linked list implementation of List and Deque.

Why it matters: It can efficiently link/unlink nodes when the node position is already known, but indexed access requires traversal.

Beginner explanation: Each node contains references to neighboring nodes. Finding an arbitrary index requires traversal.

Practical example:

```
LinkedList<String> list = new LinkedList<>();
list.add("A");
list.add("B");
list.addFirst("Start");
list.addLast("End");
```

Interview answer: LinkedList stores nodes linked to previous and next nodes. It has O(n) indexed access, so it is not automatically faster than ArrayList.

Interview comparison: ArrayList is usually preferred for general-purpose List usage because of fast indexed access and good cache locality. LinkedList is useful when deque-like operations fit the workload.

3. HashSet vs TreeSet

Definition: HashSet stores unique elements using hashing; TreeSet stores unique elements in sorted order using a tree-based structure.

Why it matters: The choice depends on whether you need ordering/sorted navigation.

Beginner explanation: HashSet is for hash-based membership; TreeSet additionally maintains sorted order.

Practical example:

```
Set<Integer> h = new HashSet<>();
h.add(30);
h.add(10);
h.add(20);

Set<Integer> t = new TreeSet<>();
```

```
t.add(30);
t.add(10);
t.add(20);

System.out.println(t); // [10, 20, 30]
```

Interview answer: HashSet is optimized for hash-based membership and does not provide sorted ordering. TreeSet maintains sorted order and typically provides O(log n) operations.

4. HashMap basics

Definition: HashMap stores key-value pairs and uses hashing to locate entries.

Why it matters: It is one of the most common Java collections and its internals are frequently asked in interviews.

Beginner explanation: A key's hash helps choose a bucket; equality then identifies the exact key in that bucket.

Practical example:

```
Map<String, Integer> marks = new HashMap<>();
marks.put("Asha", 90);
marks.put("Mekha", 95);

System.out.println(marks.get("Mekha")); // 95
```

Interview answer: HashMap maps a key to a value. It uses the key's hash to select a bucket and equality checks to identify the exact key.

5. HashMap Internals — Step by Step

Step 1 — hash: HashMap obtains a hash value from the key's hashCode and applies an internal hash-spreading operation.

Step 2 — bucket index: The hash is used to select a bucket in the table.

Step 3 — empty bucket: If no entry exists there, the new key-value entry can be placed there.

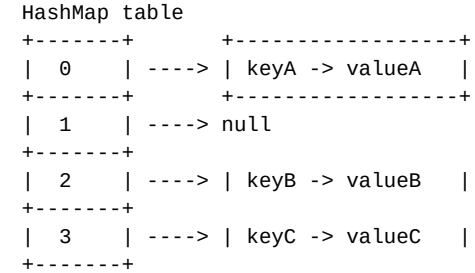
Step 4 — collision: Different keys can map to the same bucket. HashMap handles this using a linked structure and, in sufficiently large bins under specified conditions, a tree structure.

Step 5 — equality: HashMap checks key equality to determine whether the key already exists.

Step 6 — resize: When the number of entries crosses the capacity × load-factor threshold, the table is resized and entries are redistributed.

Treeification: In modern Java implementations, a heavily populated bucket can be converted from a linked-node representation into a balanced tree when table/bin thresholds are satisfied. Common OpenJDK implementation constants include TREEIFY_THRESHOLD=8, UNTREEIFY_THRESHOLD=6, and MIN_TREEIFY_CAPACITY=64. Treat these as implementation details; understand the concept first.

Conceptual picture:



Why equals() and hashCode() matter: If two keys are equal according to equals(), they must return the same hashCode. HashMap uses hashCode to find the candidate bucket and equals() to distinguish keys inside that bucket.

Important: If a mutable field used by a key's hashCode/equals changes after insertion, the entry can become difficult or impossible to retrieve using the mutated key.

6. HashMap resize and load factor

Definition: Resize expands the internal table when the number of entries crosses the threshold determined by capacity and load factor.

Why it matters: It balances memory use against collision frequency.

Beginner explanation: The commonly used default load factor is 0.75. With an initial capacity of 16, the threshold is initially around 12. When the threshold is exceeded, the table typically grows.

Practical example:

```
Map<Integer, String> map = new HashMap<>();
for (int i = 0; i < 100; i++) {
    map.put(i, "value" + i);
}
```

Interview answer: HashMap resizes when its size exceeds capacity multiplied by the load factor. Resizing costs work but reduces collision pressure.

7. ConcurrentHashMap

Definition: ConcurrentHashMap is a thread-safe map designed for concurrent access.

Why it matters: It allows multiple threads to work with the map using concurrency mechanisms without requiring one external lock around every operation.

Beginner explanation: It supports atomic compound map operations such as merge and compute and does not allow null keys or null values.

Practical example:

```
ConcurrentHashMap<String, Integer> counts = new ConcurrentHashMap<>();
counts.merge("Java", 1, Integer::sum);
counts.merge("Java", 1, Integer::sum);
```

Interview answer: ConcurrentHashMap supports concurrent map operations with internal concurrency mechanisms. It does not allow null keys or null values.

8. Fail-fast iterators

Definition: A fail-fast iterator may throw ConcurrentModificationException when the collection is structurally modified during iteration outside the iterator's supported modification mechanism.

Why it matters: It catches many accidental modifications early; it is not a synchronization guarantee.

Beginner explanation: The iterator detects certain structural changes and fails rather than silently continuing with an inconsistent traversal.

Practical example:

```
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");

for (String s : list) {
    if (s.equals("A")) {
        list.add("C"); // may throw ConcurrentModificationException
    }
}
```

Interview answer: Fail-fast means an iterator detects certain structural modifications and may throw ConcurrentModificationException. It should not be treated as a thread-safety mechanism.

9. Exceptions

Checked vs unchecked exceptions

Definition: Checked exceptions are exceptions other than RuntimeException and its subclasses that the compiler requires you to handle or declare; unchecked exceptions include RuntimeException subclasses and Error types.

Why it matters: The distinction tells you whether the compiler enforces handling or declaration.

Beginner explanation: Checked exceptions are compiler-enforced; unchecked exceptions are not required to be caught or declared by the compiler.

Practical example:

```
void readFile() throws IOException {
    FileReader r = new FileReader("data.txt");
}
```

Interview answer: Checked exceptions are compiler-enforced and commonly represent conditions callers may reasonably be expected to handle. Unchecked exceptions are generally programming errors or invalid use, such as NullPointerException or IllegalArgumentException.

try, catch and finally

Definition: try contains risky code, catch handles a matching exception, and finally is used for cleanup that should normally run whether an exception occurs or not.

Why it matters: It gives structured exception handling.

Beginner explanation: The JVM enters catch when a matching exception is thrown, and finally normally runs when control leaves the try/catch construct.

Practical example:

```
try {
    int x = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero");
} finally {
    System.out.println("Cleanup");
}
```

Interview answer: try contains code that may fail, catch handles an exception, and finally is intended for cleanup. There are control-flow edge cases such as JVM termination where finally may not execute.

throw vs throws

Definition: throw explicitly throws an exception object; throws declares that a method may propagate specified exceptions to its caller.

Why it matters: This distinction is a standard interview question.

Beginner explanation: Think: throw = action inside the method; throws = declaration in the method signature.

Practical example:

```
void validateAge(int age) {
    if (age < 18) {
        throw new IllegalArgumentException("Underage");
    }
}

void read() throws IOException {
    // caller must handle/declare the checked exception
}
```

Interview answer: throw is used inside a method to actually throw an exception. throws appears in a method signature to declare exceptions that can propagate out of the method.

Custom exceptions

Definition: A custom exception is a user-defined exception class representing an application-specific failure.

Why it matters: It makes business errors clearer than using generic exceptions.

Beginner explanation: You can extend Exception for a checked custom exception or RuntimeException for an unchecked one.

Practical example:

```
class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String message) {
        super(message);
    }
}

void withdraw(double balance, double amount)
        throws InsufficientBalanceException {
    if (amount > balance) {
        throw new InsufficientBalanceException("Insufficient balance");
    }
}
```

Interview answer: A custom exception represents a domain-specific failure. It can extend Exception for a checked exception or RuntimeException for an unchecked exception.

try-with-resources

Definition: try-with-resources automatically closes objects that implement AutoCloseable.

Why it matters: It prevents resource leaks and is safer and shorter than manually closing resources in finally.

Beginner explanation: Resources declared in the try parentheses are closed automatically when control leaves the block.

Practical example:

```
try (BufferedReader br =
         new BufferedReader(new FileReader("data.txt"))) {

    System.out.println(br.readLine());

} catch (IOException e) {
    e.printStackTrace();
}
```

Interview answer: try-with-resources automatically closes AutoCloseable resources when control leaves the try block, including when an exception occurs.

Module 2 — HashMap Drawing Drill

On paper, draw: **key** → **hashCode** → **bucket index** → **collision** → **equals()** → **value**. Then add capacity, load factor, threshold, resize, and treeification. Finally explain why a mutable key is dangerous.

Module 2 interview questions:

• Why does HashMap need both hashCode() and equals()?

• What happens when two keys have the same bucket?

• When does HashMap resize?

• What is treeification and why is it useful?

• ArrayList vs LinkedList — which one would you normally choose and why?

• HashSet vs TreeSet — what changes when sorted order is required?

• HashMap vs ConcurrentHashMap?

• What is fail-fast behavior?

- Checked vs unchecked exception?
- throw vs throws?

MODULE 3 — DAY 3: MULTITHREADING & JAVA 8

A thread is an independent path of execution inside a process. Concurrency becomes difficult because threads can access shared state at overlapping times. The central problems are visibility, atomicity and ordering.

1. Thread lifecycle

Definition: A Java thread moves through states such as NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED.

Why it matters: Knowing the states helps explain what a thread is doing when it is not currently executing.

Beginner explanation: NEW is before start; RUNNABLE means eligible to run; BLOCKED means waiting for a monitor lock; WAITING/TIMED_WAITING mean waiting for coordination or time; TERMINATED means run has completed.

Practical example:

```
Thread t = new Thread(() -> {
    System.out.println("Running");
});

System.out.println(t.getState()); // NEW
t.start();
```

Interview answer: A new Thread object starts in NEW. After start(), it becomes eligible to run. It may be blocked or waiting depending on locks and coordination, and after run() completes it becomes TERMINATED.

2. Runnable vs Thread

Definition: Runnable represents the task to execute; Thread represents the execution mechanism that runs a task.

Why it matters: Separating the task from the thread is cleaner and works naturally with executors.

Beginner explanation: Runnable describes work; Thread controls the actual thread of execution.

Practical example:

```
Runnable task = () -> {
    System.out.println("Task running");
};

Thread t = new Thread(task);
t.start();
```

Interview answer: Runnable describes work to be performed, while Thread is a thread of execution. In modern Java, prefer expressing tasks as Runnable/Callable and managing execution through executors.

3. synchronized

Definition: synchronized provides mutual exclusion and memory-visibility guarantees around a monitor-protected region.

Why it matters: It prevents multiple threads from entering a synchronized critical section guarded by the same monitor at the same time.

Beginner explanation: A synchronized instance method locks the current object's monitor; a synchronized block can lock a chosen object.

Practical example:

```
class Counter {
    private int count = 0;

    synchronized void increment() {
        count++;
    }
```

```
    int getCount() {
        return count;
    }
}
```

Interview answer: synchronized protects a critical section using an object's monitor and establishes memory-visibility guarantees between lock and unlock operations.

4. volatile

Definition: volatile tells the JVM that reads and writes of a variable must obey the Java Memory Model's visibility and ordering rules for volatile access.

Why it matters: It is useful when one thread needs to reliably observe updates made by another thread for simple state flags.

Beginner explanation: volatile does not provide mutual exclusion and does not make compound operations atomic.

Practical example:

```
class Worker {
    private volatile boolean running = true;

    void stop() {
        running = false;
    }

    void work() {
        while (running) {
            // do work
        }
    }
}
```

Interview answer: volatile provides visibility and ordering guarantees for a variable, but it does not make compound operations such as count++ atomic.

Classic trap: volatile int count; count++; is not an atomic increment. It is read → add → write, and multiple threads can interleave these steps.

5. Deadlock

Definition: Deadlock occurs when threads wait indefinitely for resources held by each other.

Why it matters: It is a classic concurrency failure and an important interview scenario.

Beginner explanation: A common pattern is two threads acquiring two locks in opposite order.

Practical example:

```
// Thread 1
synchronized (lockA) {
    synchronized (lockB) {
        // work
    }
}

// Thread 2
synchronized (lockB) {
    synchronized (lockA) {
        // work
    }
}
```

Interview answer: A deadlock can occur when there is a cycle of waiting: thread 1 holds lock A and waits for B while thread 2 holds B and waits for A.

Deadlock prevention: use consistent lock ordering, reduce lock scope, avoid unnecessary nested locks, and prefer higher-level concurrency utilities where appropriate.

6. ExecutorService

Definition: ExecutorService manages a pool of worker threads and executes submitted tasks.

Why it matters: It avoids manually creating a new thread for every task and gives you lifecycle and task-management APIs.

Beginner explanation: You submit tasks to the executor; the executor decides which worker executes them.

Practical example:

```
ExecutorService pool = Executors.newFixedThreadPool(2);

pool.submit(() -> System.out.println("Task 1"));
pool.submit(() -> System.out.println("Task 2"));

pool.shutdown();
```

Interview answer: ExecutorService separates task submission from thread management. A fixed thread pool reuses a fixed number of worker threads to execute submitted tasks.

7. wait() and notify()

Definition: wait() causes a thread to release the monitor and wait; notify()/notifyAll() wakes waiting threads associated with that monitor.

Why it matters: They are foundational thread-coordination mechanisms, though higher-level java.util.concurrent utilities are usually preferred for production code.

Beginner explanation: A thread must own the object's monitor to call these methods, and wait should normally be guarded by a while condition.

Practical example:

```
synchronized (lock) {
    while (!ready) {
        lock.wait();
    }
}

synchronized (lock) {
    ready = true;
    lock.notifyAll();
}
```

Interview answer: wait, notify and notifyAll are monitor-based coordination methods. A thread must own the object's monitor to call them, and wait should normally be used in a while loop that rechecks the condition.

Important: sleep() does not release a monitor; wait() releases the monitor associated with that wait.

8. Functional Interfaces

Definition: A functional interface has exactly one abstract method, making it usable as the target type of a lambda expression.

Why it matters: It provides the type-system foundation for lambdas and much of the Java 8 functional style.

Beginner explanation: The @FunctionalInterface annotation asks the compiler to verify the single-abstract-method rule.

Practical example:

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

```
Calculator c = (a, b) -> a + b;
System.out.println(c.add(2, 3));
```

Interview answer: A functional interface has one abstract method. Lambda expressions can provide its implementation. Common examples are Predicate, Function, Consumer and Supplier.

9. Lambda expressions

Definition: A lambda is a concise way to represent an implementation of a functional interface.

Why it matters: It reduces boilerplate when passing behavior as data.

Beginner explanation: The parameter list and arrow introduce the lambda body; the compiler infers the target functional-interface type from context.

Practical example:

```
List<String> names = List.of("Asha", "Mekha", "Anu");

names.forEach(name -> System.out.println(name));
```

Interview answer: A lambda is a compact expression of behavior that can be assigned to a functional interface or passed where such a target type is expected.

10. Streams

Definition: A Stream is a pipeline for processing elements from a data source using operations such as filter, map and collect.

Why it matters: Streams make transformations and filtering concise and composable.

Beginner explanation: filter keeps matching elements, map transforms each element, and collect produces a result container.

Practical example:

```
List<String> names =
    List.of("Asha", "Mekha", "Anu");

List<String> result = names.stream()
    .filter(name -> name.length() > 3)
    .map(String::toUpperCase)
    .collect(Collectors.toList());

System.out.println(result);
```

Interview answer: A stream processes data through a pipeline. filter selects elements, map transforms elements, and collect materializes the result into a collection or another result container.

Key idea: a Stream is not a collection. A collection stores data; a stream describes a computation over data.

11. Optional

Definition: Optional is a container type that may contain a non-null value or be empty.

Why it matters: It can make the possibility of absence explicit and reduce some null-handling mistakes.

Beginner explanation: Optional is especially useful for return values where absence is expected. It should not be treated as a universal replacement for null.

Practical example:

```
Optional<String> name = Optional.ofNullable(getName());

String result = name.orElse("Unknown");
```

Interview answer: Optional represents a value that may be present or absent. It is commonly used as a return type when absence is a normal outcome.

12. Java 8 Stream Pattern — filter → map → collect

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);

List<Integer> result = nums.stream()
    .filter(n -> n % 2 == 0)       // keep even numbers
    .map(n -> n * 10)              // transform
    .collect(Collectors.toList()); // create result

// result = [20, 40, 60]
```

Read this pipeline left to right: **source** → **filter** → **transform** → **collect**.

13. Producer–Consumer Mental Model

Producer-consumer is a classic concurrency problem. A producer creates work/data, a consumer takes work/data, and a shared buffer sits between them. The producer waits when the buffer is full; the consumer waits when it is empty. In modern Java, blocking queues are usually preferable to hand-written wait/notify code.

```
BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);

Runnable producer = () -> {
    try {
        queue.put(10);       // waits if full
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
};

Runnable consumer = () -> {
    try {
        int value = queue.take(); // waits if empty
        System.out.println(value);
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
};
```

Interview point: Explain the shared buffer, blocking condition, thread coordination, and why a thread-safe queue is useful.

Module 3 — Interview Questions

• Explain the Java thread lifecycle.

• Why is Runnable usually preferred over extending Thread?

• What does synchronized guarantee?

• volatile vs synchronized?

• Why does volatile not make count++ thread-safe?

• What is deadlock? Give a two-lock example.

• How can deadlock be prevented?

• Why use ExecutorService?

• Difference between wait() and sleep()?

• What is a functional interface?

• What do filter(), map() and collect() do?

• Is a Stream a collection?

• Why use Optional?

FINAL INTERVIEW CHEATSHEET

These are speaking versions. Understand the detailed sections first; then practice saying these without reading.

Four pillars

Encapsulation controls access to object state; inheritance allows a subclass to reuse/extend a superclass; polymorphism lets one general reference work with different implementations; abstraction exposes essential behavior while hiding implementation details.

Abstract class vs interface

Use an abstract class when closely related classes need shared state or implementation. Use an interface when you want a contract or capability that multiple classes can implement.

Overloading vs overriding

Overloading uses the same method name with different parameter lists and is resolved at compile time. Overriding occurs when a subclass provides a new implementation of an inherited method and supports runtime polymorphism.

static

A static member belongs to the class rather than a particular object. Static fields are shared among instances, and static methods cannot directly use instance state through this.

final

A final variable cannot be reassigned, a final method cannot be overridden, and a final class cannot be extended.

this vs super

this refers to the current object. super refers to the immediate superclass and is used to access parent members or call a parent constructor.

String pool

The String Pool stores canonical strings so identical literals can be reused. String literals use the pool, while new String creates a distinct object.

String immutability

A String cannot be modified after it is created. Operations that appear to change it create another String, which makes sharing and pooling safe.

String vs StringBuilder vs StringBuffer

String is immutable. StringBuilder is mutable and generally preferred for repeated string construction in a single-threaded context. StringBuffer is mutable and synchronizes its methods.

== vs equals

For objects, == checks reference identity while equals checks logical equality according to the class implementation. For String, equals compares character content.

HashMap

HashMap uses a key's hash to choose a bucket and equals to identify the exact key. Collisions can form linked structures and, under certain thresholds, tree structures. It resizes when its threshold is exceeded.

ArrayList vs LinkedList

ArrayList uses a dynamic array and gives fast indexed access. LinkedList uses linked nodes and is better suited to certain deque or known-node insertion/removal operations. ArrayList is the common default.

HashSet vs TreeSet

HashSet provides uniqueness through hashing without sorted ordering guarantees. TreeSet maintains sorted order and typically gives logarithmic operations.

Checked vs unchecked

Checked exceptions are compiler-enforced and must be caught or declared. Unchecked exceptions are RuntimeException subclasses and are not compiler-enforced in that way.

throw vs throws

throw actually throws an exception object. throws declares exceptions that a method may propagate to its caller.

synchronized vs volatile

synchronized protects critical sections and provides mutual exclusion plus memory-visibility guarantees. volatile provides visibility and ordering for a variable but does not make compound operations atomic.

Deadlock

Deadlock is indefinite waiting caused by threads holding resources while waiting for resources held by each other. Consistent lock ordering and smaller critical sections are common prevention techniques.

ExecutorService

ExecutorService manages worker threads and executes submitted tasks, separating task submission from thread creation and lifecycle management.

Lambda and functional interface

A functional interface has one abstract method, and a lambda provides its implementation concisely.

Streams

Streams provide a pipeline for processing data. filter selects elements, map transforms them, and collect materializes a result.

Optional

Optional represents a value that may be present or absent and makes that possibility explicit, commonly for method return values.

3-Day Pass Marks

Day 1: Explain each listed OOP/string topic aloud in under 90 seconds without pausing. Then solve String Pool and ==/equals prediction questions without running the code.

Day 2: Draw a HashMap on paper from memory: key → hash → bucket → collision → equals → value, then add capacity, load factor, threshold, resize, and treeification.

Day 3: Write either a producer-consumer example or a deadlock example without help. Explain the synchronization/coordination mechanism and the failure mode.

Final standard: Do not judge success by whether you can repeat a definition. You pass when you can explain *why* the mechanism behaves that way and predict a small code snippet's output.